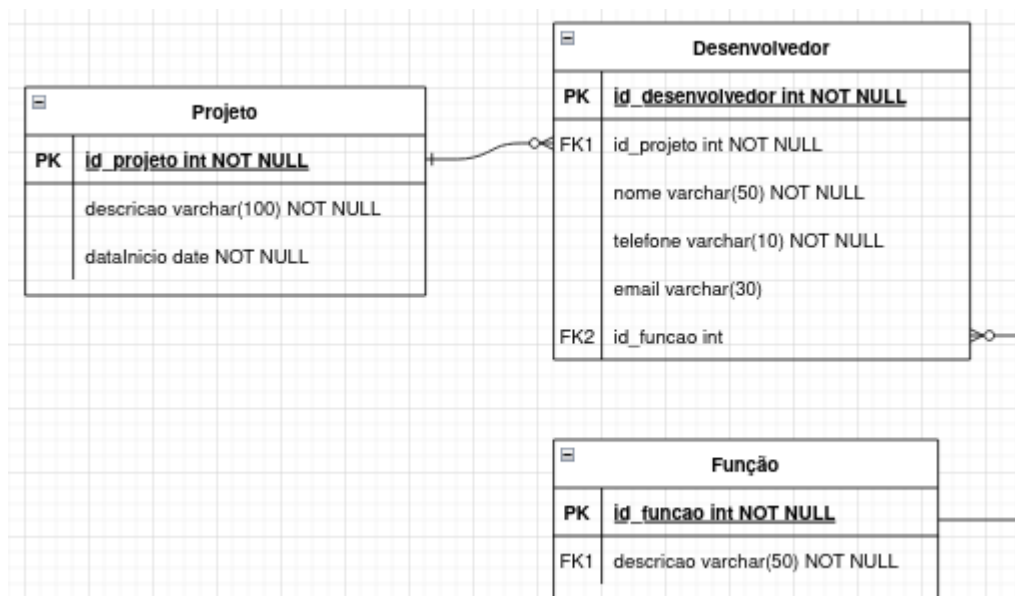


INTEGRIDADE REFERENCIAL NO MYSQL

Quando uma tabela referencia outra por meio de chave estrangeira, precisamos “dizer” ao SGBD o que deverá ser feito caso um registro da tabela A que é referenciado por um registro da tabela B for excluído ou alterado.

Em resumo o funcionamento é o seguinte: quando uma chave primária de uma tabela é excluída ou alterada, o SGBD procura por chaves estrangeiras que referenciem essa chave primária. Se encontrar, verifica nas definições da tabela que possui a chave estrangeira qual ação deve ser tomada.

Exemplo: no modelo a seguir, existe uma chave estrangeira na tabela “Desenvolvedor” que permite sabermos de quais projetos o desenvolvedor participa. O que deve acontecer na tabela “Desenvolvedor” caso um projeto que possua desenvolvedores associados, seja excluído? Ou, o que deve acontecer na tabela “Desenvolvedor” caso um projeto que possua desenvolvedores associados, tenha o valor de sua chave primária (id_projeto) alterado.?



O MySQL implementa as restrições de integridade CASCADE, RESTRICT, SET NULL e SET DEFAULT.

- **CASCADE:** ao remover um registro da tabela referenciada pela chave estrangeira os registros relacionados àquele removido serão eliminados em todas as tabelas relacionadas.
- **RESTRICT:** não permite a remoção de registros que possuam relacionamentos em outras tabelas.
- **SET DEFAULT:** atribui o valor *default* para as chaves estrangeiras cujos registros relacionados foram excluídos. O valor default é o atribuído ao campo no momento da criação da tabela, se não houver um valor default definido, o campo será setado para null.
- **SET NULL:** atribui o valor *null* para as chaves estrangeiras cujos registros relacionados foram excluídos.

As restrições acima podem ser utilizadas também caso haja alteração no valor da chave primária de uma tabela que é referenciada por outras.

Administração para Banco de Dados
Prof.: Aline Gomes Cordeiro

O exemplo abaixo ilustra algumas tabelas que utilizam regras de integridade:

```
CREATE TABLE projeto (  
    id_projeto int not null auto_increment primary key,  
    descricao varchar(100) not null,  
    dataInicio date not null  
)  
  
CREATE TABLE desenvolvedor (  
    id_desenvolvedor int not null auto_increment primary key,  
    id_projeto int not null,  
    nome varchar(50) not null,  
    telefone varchar(10) not null,  
    email varchar(30),  
    id_funcao int not null,  
    index idx_funcao (id_funcao),  
    index idx_projeto (id_projeto),  
    foreign key (id_projeto) references projeto (id_projeto) ON DELETE SET NULL ON UPDATE  
    CASCADE,  
    foreign key (id_funcao) _____  
)
```

No exemplo acima, caso um projeto que possua desenvolvedores associados seja excluído, as referências a este projeto na tabela “desenvolvedor”, devem ser setadas para null. Caso o “id_projeto” de algum projeto seja alterado, suas referências na tabela “desenvolvedor” devem ser modificadas para o mesmo valor.

É muito importante definir explicitamente como as chaves estrangeiras devem se comportar se os dados das tabelas de origem forem excluídos ou alterados.

Complete a definição da FK id_funcao. O SGBD não deve permitir a exclusão nem a alteração de “id_funcao” de funções que possuam desenvolvedores associados.

É importante ressaltar que o FOREIGN KEY não cria automaticamente um índice na(s) coluna(s) referenciada(s). Assim, é necessário criar explicitamente um índice nas colunas que serão chaves estrangeiras.

ÍNDICES

Os índices aceleram a recuperação dos dados. Por exemplo, imagine que você compre um livro de 800 páginas para suas pesquisas acadêmicas e este não apresente em seu conteúdo um índice reportando o seu conteúdo. Uma pesquisa talvez não fosse tão pavorosa, mas se você precisar de várias pesquisas, seria muito desagradável ficar horas procurando o conteúdo que deseja estudar. Por outro lado, um livro que apresente um índice de suas abordagens, se faz muito mais fácil e torna as pesquisas até prazerosas, pois teremos condição de irmos direto ao ponto que queremos.

Os índices são utilizados para encontrar registros com um valor específico de uma coluna rapidamente. Sem um índice o MySQL tem de iniciar com o primeiro registro e depois ler através de toda a tabela até que ele encontre os registros relevantes. Quanto maior a tabela, maior será o custo. Se a tabela possui um índice para as colunas em questão, o MySQL pode rapidamente obter uma posição para procurar no meio do arquivo de dados sem ter que varrer todos os registros.

Todos os índices do MySQL (PRIMARY, UNIQUE e INDEX) são armazenados em árvores Binárias.

ENTENDO OS ÍNDICES

Quando temos uma tabela e executamos uma operação de SELECT sobre ela, filtrando por um ou vários campos, o gerenciador do banco efetua uma ação chamada "TABLE SCAN". Essa ação consiste em percorrer toda a tabela, avaliando cada registro. Caso o registro atenda às condições definidas no filtro, ele é incluído no conjunto de retorno, senão, é apenas desconsiderado.

A figura a seguir ilustra uma pesquisa desse tipo, filtrando uma tabela pelo campo "Codigo", onde seu valor seja "3".

	Codigo	Nome
Codigo = 3? Não. Desconsidere. →	1	Joao
Codigo = 3? Não. Desconsidere. →	2	Antonio
Codigo = 3? Sim. Retornar registro. →	3	Maria

No exemplo acima a tabela continha apenas três registros e a consulta foi facilmente concluída, porém, imagine algo semelhante em uma tabela com milhares de linhas.

Para melhorar essas consultas, utilizamos ÍNDICES, objetos do banco de dados que facilitam a organização e consulta de uma tabela, "indexando-a" por uma de suas colunas.

Quando criamos um índice em uma coluna, o gerenciador do banco ordena a tabela por essa coluna e a partir de então os filtros (sobre essa coluna) são feitos através de uma busca binária.

Na busca binária, a tabela sofre divisões sucessivas até que o registro desejado seja localizado. Inicialmente a tabela é "dividida" na metade, onde os registros com os menores valores no campo indexado ficam na parte superior, e os registros com os maiores valores ficam na parte inferior. O gerenciador verifica então em que parte está contido o valor pesquisado, se na primeira metade ou na segunda. Isso é facilmente feito comparando o valor pesquisado com o do registro intermediário. Se o valor pesquisado for maior que o do registro intermediário, o registro desejado encontra-se na segunda metade, caso contrário, encontra-se na primeira metade. Na figura a seguir temos uma ilustração desse processo.

Administração para Banco de Dados
Prof.: Aline Gomes Cordeiro

Codigo	Nome
1	Joao
2	Antonio
3	Maria
4	Carlos
5	Francisca
6	Pedro
7	Jose
8	Maria
9	Xavier
10	Joaquim

} Primeira metade

} Segunda metade

Por exemplo, se pesquisássemos pelo registro de Codigo = 3, seria avaliado se esse valor é maior ou menor que 5. Como o valor é menor, indica que o registro procurado encontra-se na primeira metade.

Em seguida, o processo é repetido apenas na metade em que se encontra o registro pesquisado, dividindo-a na metade e avaliando novamente em que parte encontra-se o que se deseja.

Note que na operação de table scan, todos os registros seriam lidos e com a utilização de índices, na primeira divisão já se descarta metade dos registros da tabela. Para tabelas pequenas a diferença dificilmente é percebida, mas quando se trabalha com grandes volumes de dados, essa simples mudança pode melhorar e muito a velocidade de retorno das consultas.

QUANDO NÃO CRIAR ÍNDICES:

Considere as seguintes observações antes de criar índices. Quando colunas indexadas são modificadas, o SGBD desloca recurso internamente para manter esses índices atualizados e associados;

Quando o atributo contém grande quantidade de dados duplicados, índices apresentam mais custo que benefícios. Exemplo: atributos do tipo “gênero” não devem ser indexados.

SINTAXE:

Na criação da tabela:

```
CREATE TABLE notas (  
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,  
    aluno_id INT NOT NULL,  
    cursos_id INT NOT NULL,  
    date DATE NOT NULL,  
    nota DOUBLE NOT NULL,  
    INDEX idx_cursos (cursos_id),  
    FOREIGN KEY (aluno_id) REFERENCES aluno(id) ON DELETE CASCADE,  
    FOREIGN KEY (cursos_id) REFERENCES cursos(id) ON DELETE RESTRICT  
);
```

Administração para Banco de Dados

Prof.: Aline Gomes Cordeiro

Depois que a tabela já está criada:

```
CREATE INDEX idx_CLIENTES ON CLIENTES(Codigo);
```

REFERÊNCIAS:

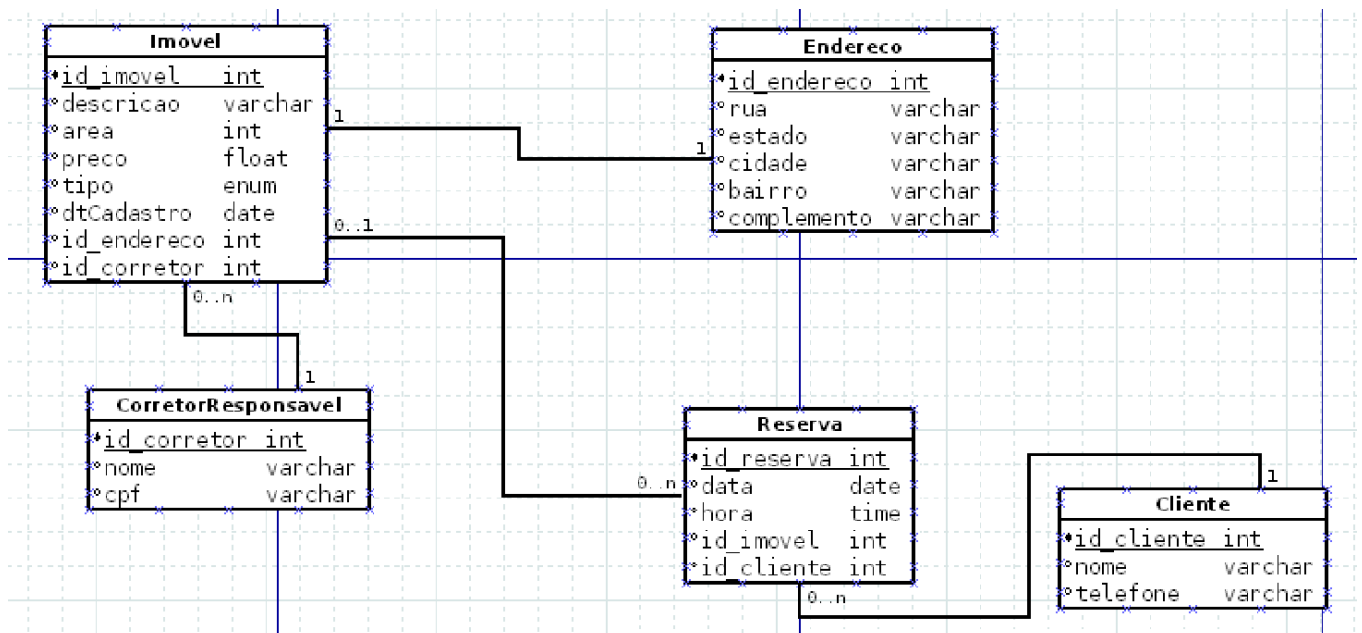
<http://www.devmedia.com.br/entendendo-e-usando-indices-parte-1/6567#ixzz2McRNUrwG>

<http://www.devmedia.com.br/implementando-integridade-referencial-no-mysql/1996>

<http://dev.mysql.com/doc/refman/5.1/en/create-index.html>

Exercício

A imobiliária ABC mantém um banco de dados relativo aos imóveis que possui. Cada imóvel possui um corretor responsável. Os imóveis podem ser reservados para os clientes.



- 1) Crie o banco de dados e as tabelas conforme o modelo. Crie índices para as chaves estrangeiras e defina as restrições de integridade das tabelas para atender aos requisitos a seguir:
 - a) Se endereço de um imóvel for excluído, o SGBD deve setar a referência desse endereço na tabela imóvel para null.
 - b) O cadastro de um corretor não pode ser excluído se houver imóveis sob sua responsabilidade.
 - c) Se um imóvel for excluído as reservas associadas a ele também devem ser excluídas.
 - d) Se um cliente que possui reservas for excluído todas as suas reservas também devem ser excluídas.
- 3) Insira dados nas tabelas;
- 4) Faça exclusões de forma a validar a restrições de integridade definidas;